PASSWORD LOCK SYSTEM USING VERILOG HDL

MINI PROJECT PRESENTATION BY :

GADI SURESH
23A91A0417
ECE
Aditya Engineering College

What is a Password Locking System?

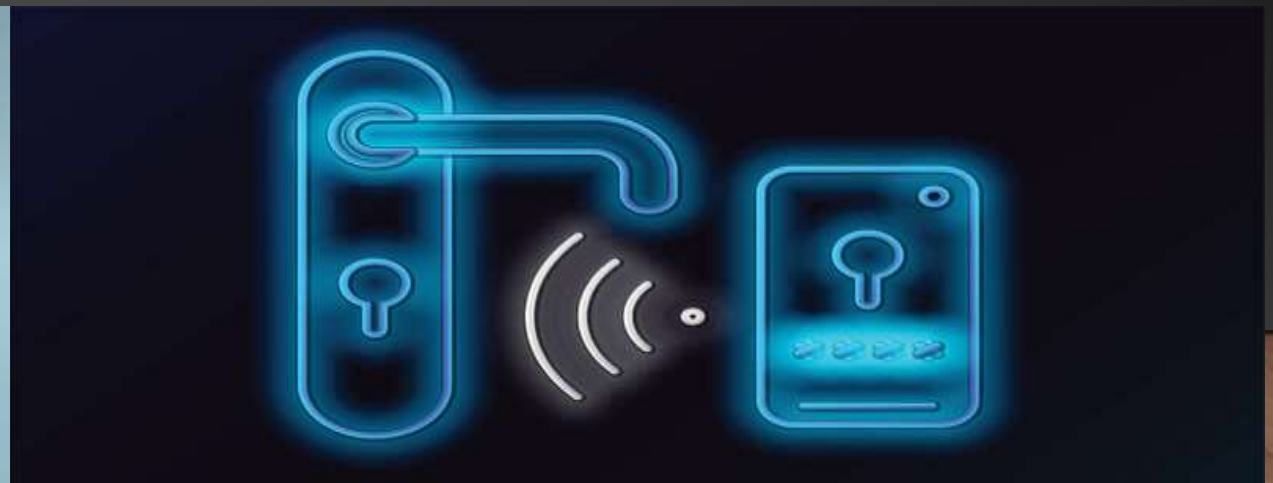
- A digital access control mechanism that verifies user identity using a password.
- Compares the entered password with a stored or predefined password.
- Grants access if the entered password matches the stored one.
- Denies access and increments a failure counter if the password is incorrect.
- Triggers an alarm after a defined number of incorrect attempts (typically 3).
- Can include features like password change and master password override for security and recovery.
- Commonly used in digital lockers, smart home systems, lab access controls, and IoT devices.

Extra Features:

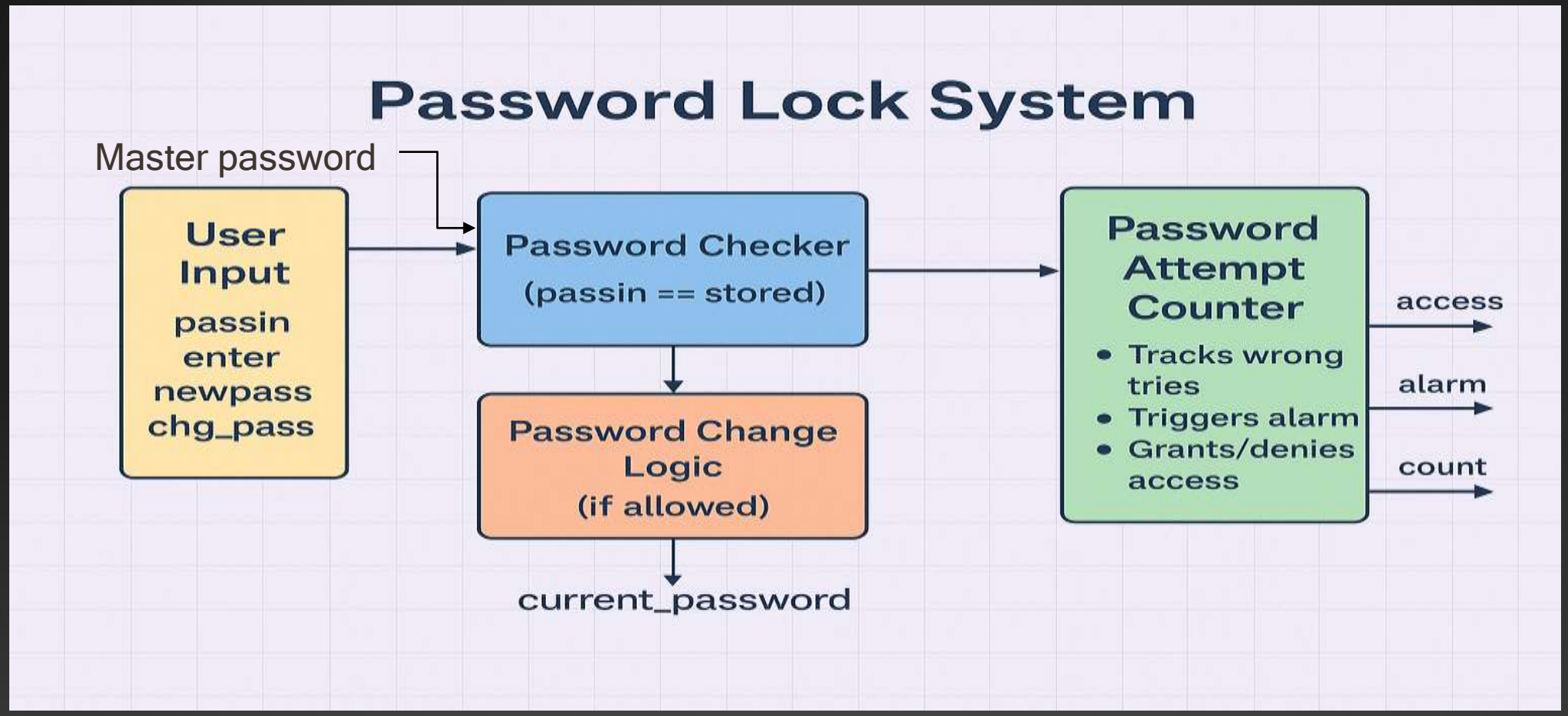
Feature	Previous Version	Updated Version
Password Changing Option	Not available	Can change password after correct entry
Default Password	Not available	Resetting returns password to default value
Master Password	Not available	Added master password for emergency access
Alarm Triggering	limited alarm system	Improved alarm triggering on failed attempts
Remaining Attempts Display	No user feedback	Shows messages with remaining attempts
Reset Functionality	Limited	Resets password to default and clears attempts

Why this Project?

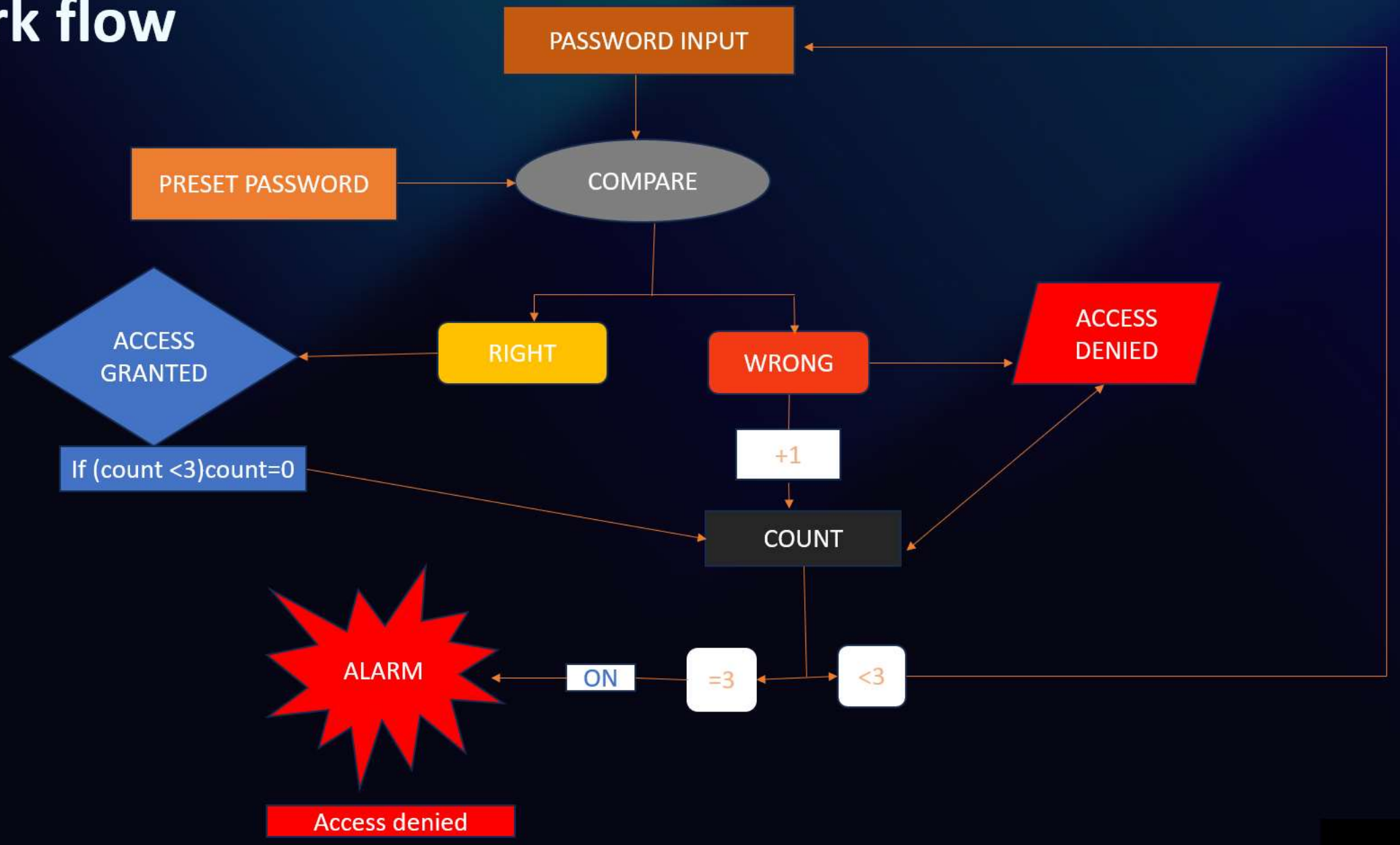
- Password protection is a **fundamental requirement** in modern digital systems.
- Helps us understand **secure authentication techniques** using Verilog HDL.
- Useful for **real-time applications** like smart locks, IoT devices, and security panels.
- Combines **practical hardware design** with **logical control and verification**.
- Encourages **modular design thinking** and simulation-based validation.
- Demonstrates the integration of **access logic, alarm triggers, and user input handling**.



Block Diagram :



Work flow


PASSWORD INPUT
PRESET PASSWORD
COMPARE
ACCESS GRANTED
RIGHT
WRONG
ACCESS DENIED
If (count <3)count=0
+1
COUNT
ALARM
ON
=3
<3
Access denied

Verilog modules overview :

1.Top Module (password_lock_system.v)

```
// Top module: password_lock_system
module password_lock_system(
    input reset,
    input clk,
  input [15:0] passin,      // Entered password
  input [15:0] newpass,     // New password to set
    input enter,             // Login attempt signal
    input chg_pass,          // Change password signal
    output access,
    output alarm,
    output [1:0] count       // Number of wrong attempts
);
  reg [15:0] current_password;
    wire is_correct;
  wire [15:0] master_password = 16'habcd;  // Master password
    initial begin
        current_password = 16'h1234;  // Default initial password
    end

    assign is_correct = ((passin == current_password) || (passin == master_password));

    // Password change logic
    always @(posedge clk or negedge reset) begin
        if (!reset)
            current_password <= 16'h1234;  // Reset to default
        else if (chg_pass && is_correct)
            current_password <= newpass;  // Change password only if authenticated
    end
    password_attempt_counter counter (.clk(clk),.enter(enter),.correct(is_correct),.rstn(reset), .cnt(count), .access(access), .alarm(alarm));
endmodule
```

2. Counter module (password_attempt_counter.v) :

```
// Password attempt counter module
module password_attempt_counter(
    input clk,
    input enter,
    input correct,
    input rstn,
    output reg [1:0] cnt,
    output reg access,
    output reg alarm
);
    parameter N = 4;

  always @(posedge clk or negedge rstn) begin
        if (!rstn) begin
            cnt <= 0;
            access <= 1;
            alarm <= 0;
        end else if (enter) begin
            if (correct) begin
                cnt <= 0;
                access <= 1;
                alarm <= 0;
            end else if (cnt == N - 2) begin
                cnt <= cnt + 1;
                access <= 0;
                alarm <= 1;
            end else if (cnt == N - 1) begin
                access <= 0;
                alarm <= 1;
            end else begin
                cnt <= cnt + 1;
                access <= 0;
                alarm <= 0;
            end
        end
    end
endmodule
```

1. Tracking of wrong attempts
2. Triggers alarm after 3 failures attempts
3. Grant acces if password is correct
4. On Reset Default password set to1234

Testbench (tb_password_lock_system.v) :

1. Simulate multiple password attempt
2. check acess, alarm and count signal
3. uses the clock and reset logic to test real behaviour

```
module tb_password_lock_system;
  reg clk, reset;
  reg [15:0] passin, newpass;
  reg enter, chg_pass;
  wire access, alarm;
  wire [1:0] count;
  // Instantiate the design
  password_lock_system dut
(.reset(reset),.clk(clk),.passin(passin),.newpass(newpass),.enter(enter),.chg_pass(chg_pass),.access(access),.alarm(alarm),.count(count));
// attempt counter module instantiate
reg [1:0] attempt_count;
remaining_attempts_display disp(.count(attempt_count));
// remaining_attempts_display.sv or inside testbench
module remaining_attempts_display(
    input [1:0] count
);
    always @(*) begin
        case (count)
          2'd0: $display("    System unlocked successfully.");
            2'd1: $display("  Incorrect password. You have 2 attempts left.");
            2'd2: $display("  Incorrect password. You have 1 attempt left.");
            2'd3: $display("  Limit reached! Alarm triggered.\n");
            default: $display("   Invalid count value.\n");
        endcase
    end
endmodule
```

Testbench (tb_password_lock_system.v) :

```
  // Clock generation
  always #5 clk = ~clk;
  initial begin
    clk = 0;
    reset = 0;
    enter = 0;
    chg_pass = 0;
    passin = 16'h0000;
    newpass = 16'h0000;

    // Apply reset
    #10 reset = 1;

    // Test 1: Correct default password (1234)
    $display("\n  Feature 1 :Correct password ");


    #10 passin = 16'h1234; enter = 1;
    #10 enter = 0;
    $display("    Test 1: reset=%b  passin=%h  current=%h  master=%h  access=%b  alarm=%b  count=%h ",
             reset, passin, dut.current_password, dut.master_password, access, alarm, count);
    attempt_count = count;

//alaram triggering condtion
    #10;$display("\n  Feature 2 : alaram triggering condtion ");
    // Test 2.1
    #10 passin = 16'h1516; enter = 1;
    #10 enter = 0;
    $display("    Test 2.1: reset=%b  passin=%h  current=%h  master=%h  access=%b  alarm=%b  count=%h ",
             reset, passin, dut.current_password, dut.master_password, access, alarm, count);
    attempt_count = count;
    // Test 2.2
    #10 passin = 16'h2127; enter = 1;
    #10 enter = 0;
    $display("    Test 2.2: reset=%b  passin=%h  current=%h  master=%h  access=%b  alarm=%b  count=%h ",
             reset, passin, dut.current_password, dut.master_password, access, alarm, count);
    attempt_count = count;
    // Test 2.3
    #10 passin = 16'h348; enter = 1;
    #10 enter = 0;
    $display("    Test 2.3: reset=%b  passin=%h  current=%h  master=%h  access=%b  alarm=%b  count=%h ",
             reset, passin, dut.current_password, dut.master_password, access, alarm, count);
    attempt_count = count;
```

Testbench (tb_password_lock_system.v) :

```
// Test 3: Change password from 1234 to 4321 (valid)
#10 $display("    Feature 3 : Password changing option using current password ");
#10 passin = 16'h1234; newpass = 16'h4321; chg_pass = 1; enter = 1;
#10 enter = 0;
$display("    Test 3.1 : current password changed to '4321'");


// Test 4: Login with new password (4321)
#10 passin = 16'h4321; enter = 1;
#10 enter = 0;
$display("    Test 3.2: reset=%b  passin=%h  current=%h  master=%h  access=%b  alarm=%b  count=%h ",
         reset, passin, dut.current_password, dut.master_password, access, alarm, count);
attempt_count = count;


// Test 5: Invalid password change with wrong passin
#10 passin = 16'h1234; enter = 1;
#10 enter = 0; chg_pass = 0;
$display("    Test 3.3: reset=%b  passin=%h  current=%h  master=%h  access=%b  alarm=%b  count=%h ",
         reset, passin, dut.current_password, dut.master_password, access, alarm, count);
attempt_count = count;


// Test 6: Change password using master password (4321) to 9866
#10; $display("\n Feature 4 : changing current password  with master password ");
#10 passin = 16'habcd; newpass = 16'h9866; chg_pass = 1; enter = 1;
#10 enter = 0; chg_pass = 0;
$display("    Test 4.1:   current password changed to '9866'" );


// Test 7: Login with master password
#10; $display("\n  unlocking system  with master password ");
#10 passin = 16'habcd; enter = 1;
#10 enter = 0;
$display("    Test 4.2: reset=%b  passin=%h  current=%h  master=%h  access=%b  alarm=%b  count=%h ",
         reset, passin, dut.current_password, dut.master_password, access, alarm, count);
attempt_count = count;
```

Testbench (tb_password_lock_system.v) :

```
    //reset condition
    #10 $display("\n  Feature 5 : on reset condition : ");
    #10 reset = 0;
    #10 reset =1;
    #10 reset = 0;

    $display("    Test 5.1: reset=%b  passin=%h  current=%h  master=%h  access=%b  alarm=%b  count=%h ",
             reset, passin, dut.current_password, dut.master_password, access, alarm, count);
    #10;attempt_count = count;
    $display("    System unlocked successfully.");

    #10 $display("\n Simulation Complete.");
    $finish;
  end
endmodule
```

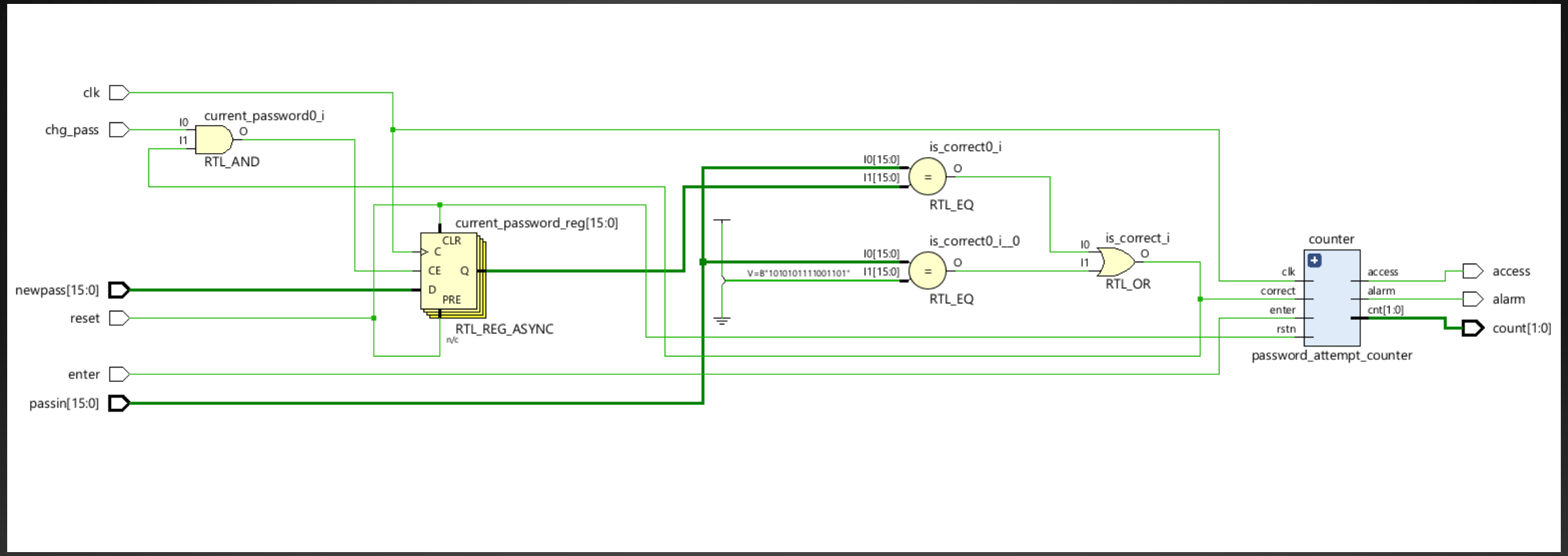
Output :

```
Feature 1 :Correct password
Test 1: reset=1  passin=1234  current=1234  master=abcd  access=1  alarm=0  count=0
System unlocked successfully.

Feature 2 : alaram triggering condtion
Test 2.1: reset=1  passin=1516  current=1234  master=abcd  access=0  alarm=0  count=1
Incorrect password. You have 2 attempts left.
Test 2.2: reset=1  passin=2127  current=1234  master=abcd  access=0  alarm=0  count=2
Incorrect password. You have 1 attempt left.
Test 2.3: reset=1  passin=0348  current=1234  master=abcd  access=0  alarm=1  count=3
Limit reached! Alarm triggered.

Feature 3 : Password changing option using current password
Test 3.1 : current password changed to '4321'
Test 3.2: reset=1  passin=4321  current=4321  master=abcd  access=1  alarm=0  count=0
System unlocked successfully.
Test 3.3: reset=1  passin=1234  current=4321  master=abcd  access=0  alarm=0  count=1
Incorrect password. You have 2 attempts left.

Feature 4 : changing current password  with master password
Test 4.1:   current password changed to '9866'

 unlocking system  with master password
Test 4.2: reset=1  passin=abcd  current=9866  master=abcd  access=1  alarm=0  count=0
System unlocked successfully.

Feature 5 : on reset condition :
Test 5.1: reset=0  passin=abcd  current=1234  master=abcd  access=1  alarm=0  count=0
System unlocked successfully.
```

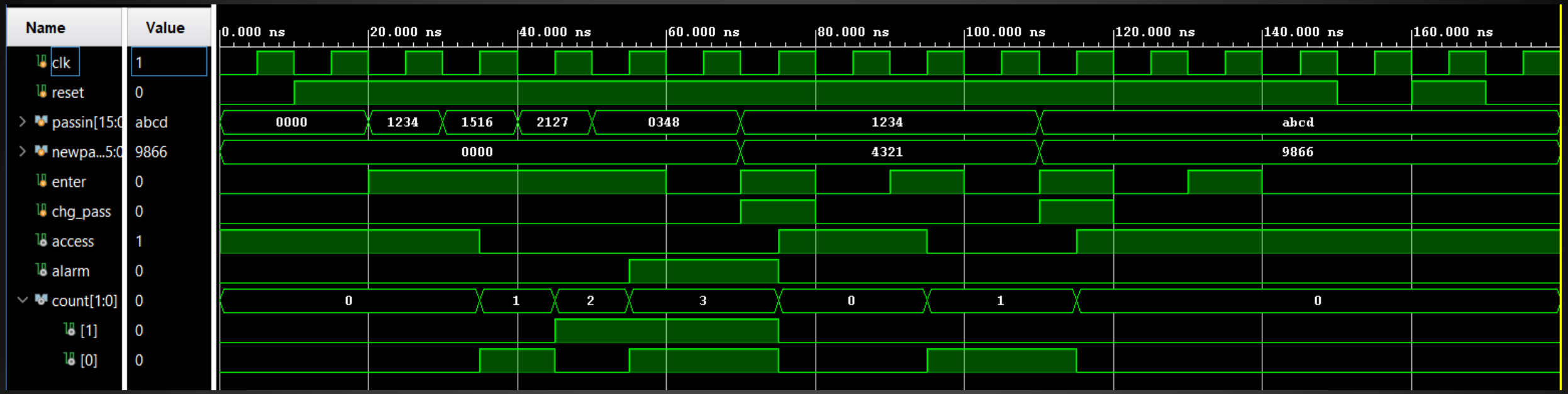
Output Analysis :

Attempt	Inputs	Current Password	Master Password	Explanation	Outputs
1	reset=1, passin=1234	1234	abcd	Correct password entered on first try; access is granted immediately.	access=1, alarm=0, count=0
2	reset=1, passin=1516	1234	abcd	Wrong password attempt; system increments the failure count.	access=0, alarm=0, count=1
3	reset=1, passin=2127	1234	abcd	Another incorrect password; system counts second failed attempt.	access=0, alarm=0, count=2
4	reset=1, passin=0348	1234	abcd	Third failed attempt; alarm is triggered due to security limit reached.	access=0, alarm=1, count=3
5	reset=1, passin=1234	4321(changed)	abcd	Password has been updated; correct password entered again.	
6	reset=1, passin=1234	4321	abcd	Incorrect password based on new updated value; first failed attempt recorded.	access=0, alarm=0, count=1
7	reset=1, passin=abcd	9866(changed)	abcd	Master password entered; system grants access regardless of current password.	
8	reset=0	1234(reseted)	abcd	System in test mode (reset=0); valid password still unlocks system.	access=1, alarm=0, count=0

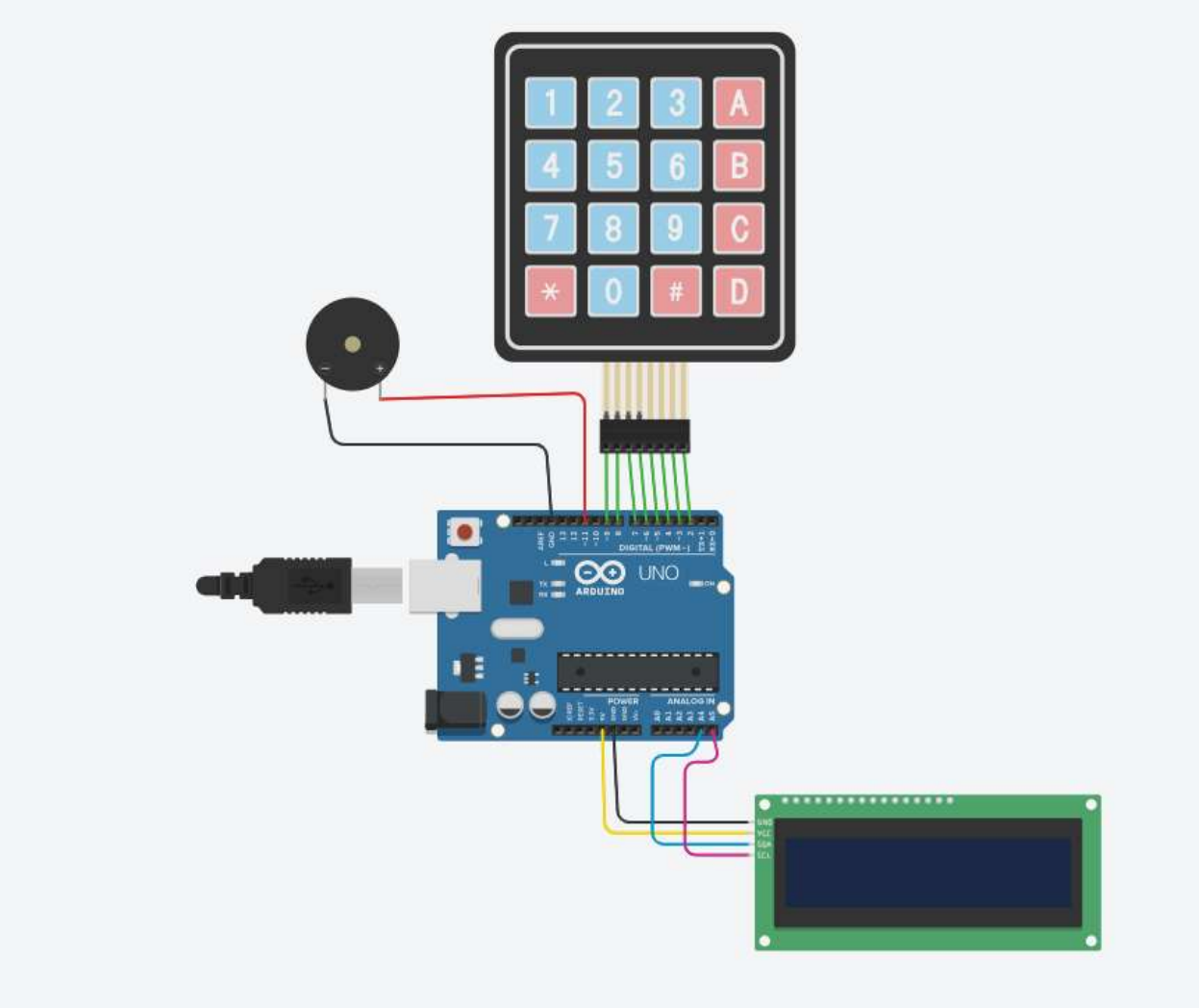
Schematic :


clk
chg_pass
current_password0_i
I0
I1
O
RTL_AND
current_password_reg[15:0]
CLR
C
CE
Q
D
PRE
RTL_REG_ASYNC
n/c
newpass[15:0]
reset
enter
passin[15:0]
is_correct0_i
I0[15:0]
I1[15:0]
=
O
RTL_EQ
is_correct0_i__0
I0[15:0]
V=B"1010101111001101"
I1[15:0]
=
O
RTL_EQ
is_correct_i
I0
I1
O
RTL_OR
counter
clk
correct
enter
rstn
access
alarm
cnt[1:0]
password_attempt_counter
access
alarm
count[1:0]

Wave forms :


Name
Value
clk
1
reset
0
passin[15:0
abcd
newpa...5:0
9866
enter
0
chg_pass
0
access
1
alarm
0
count[1:0]
0
[1]
0
[0]
0
0.000 ns
20.000 ns
40.000 ns
60.000 ns
80.000 ns
100.000 ns
120.000 ns
140.000 ns
160.000 ns
0000
1234
1516
2127
0348
1234
abcd
0000
4321
9866
0
1
2
3
0
1
0

Working Demo With Aurdino Uno :



Applications :

Where can it be used ?

- Smart Home Security Systems
- Digital Lockers
- Lab Entry Control Systems
- IoT Authentication Devices
- Basic Digital Authentication

Conclusion :

1. Designed a working **Password Locking System in Verilog**
2. Implemented alaram, counter and access logic
3. Successfully simulated the behavior with real-world scenarios
4. Demonstrates practical understanding of digital systems

THANK YOU